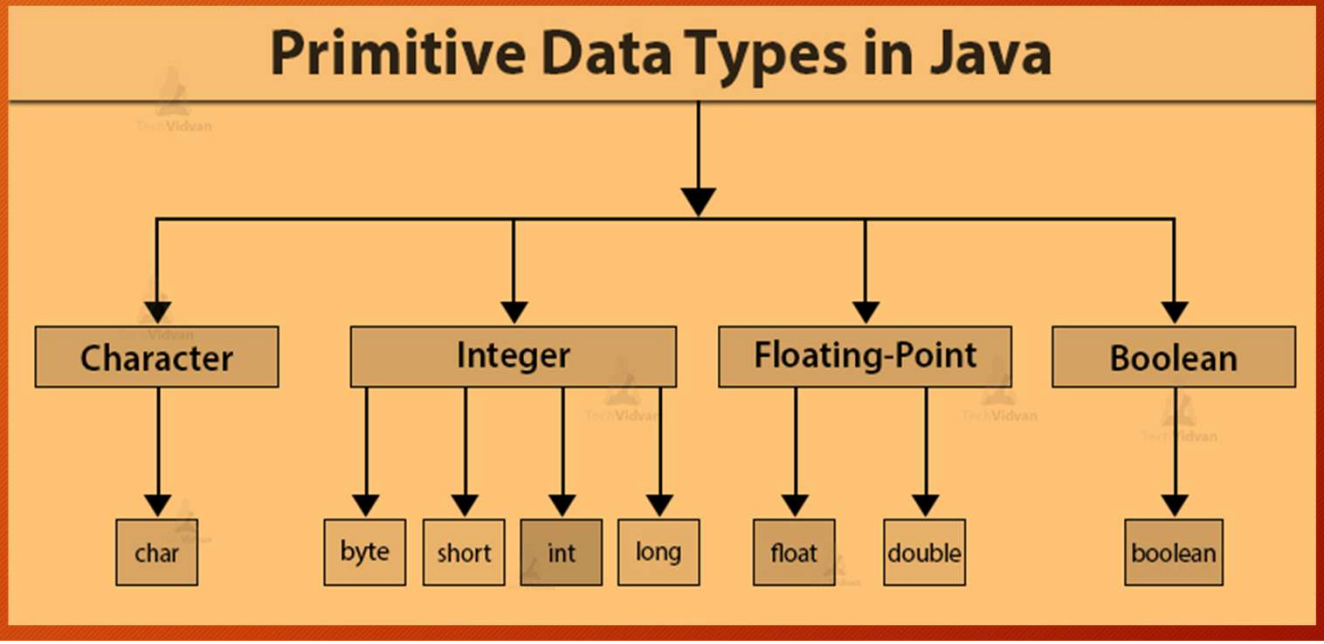
Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.

Java Primitive Data Types

- Java is a statically-typed programming language. It means, all variables must be declared before its use. That is why we need to declare variable's type and name.



Data Types

Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

1. Byte

Syntax:
byte myByte1 = -100 ;
byte myByte2 = 25 ;

- The byte data type is mostly used in the large arrays which need memory savings. It saves memory as it is 4 times smaller than an int (integer). Sometimes, it can also be used in place of the “int” data type, when the value is very small.

Size:	Values:	Default Value:	Range:
1 byte(8 bits)	Positive or negative whole numbers.	0	-128 to 127

```
public class ByteDataType
{
        public static void main(String args[])
        {
                byte myByte1,myByte2;
                myByte1 = 127;
                myByte2 = -48;
                System.out.println("Byte 1: " +myByte1);
                System.out.println("Byte 2: " +myByte2);
                myByte1++; // Looping back within the range
                System.out.println("Incremented Value of myByte1: " +myByte1);
        }
}
```

OUTPUT

```
Byte 1: 127
Byte 2: -48
Incremented Value of
myByte1: -128
```

2. Short

Syntax:
short myShort = 6000 ;

- Similar to the byte data type, a short data type is also used to save memory in large arrays. The short data type is a 16-bit signed 2's complement integer. It is half of an int (integer).

Size:	Values:	Default Value:	Range:
2 bytes (16 bits)	Positive or negative whole numbers.	0	-32768 to 32767

```
public class ShortDataType
{
        public static void main(String args[])
        {
                short myShort = 6000;
                System.out.println("myShort: " + myShort);
        }
}
```

OUTPUT

```
myShort: 6000
```

3. int

Syntax:
int myNum = 700000 ;

- Generally, we prefer to use int data type for integral values unless if there is no issue with memory. The int data type is a 32-bit integer.

Size:	**Values:**	**Default Value:**	**Range:**
4 bytes (32 bits)	Positive or negative whole numbers.	0	– 2,147,483,648 (-231) to 2,147,483,647 (231 -1)

```
public class IntDataType
{
        public static void main(String args[])
        {
                int myNum1, myNum2, result;
                myNum1 = -7000;
                myNum2 = 90000;
                result = myNum1 + myNum2;
                System.out.println("Number 1: " +myNum1);
                System.out.println("Number 2: " +myNum2);
                System.out.println("Number 1 + Number 2: " +result);
        }
}
```

OUTPUT

```
Number 1: -7000
Number 2: 90000
Number 1 + Number 2: 83000
```

4. long

Syntax:
long myLong = 11000000000L ;

- The long data type is a 64-bit signed 2's complement integers. It is used when int data type cannot hold a value. It is 2 times larger than int(integer). We must use **L** at the end of the value.

Size:	**Values:**	**Default Value:**	**Range:**
8 bytes (64 bits)	Positive or negative whole numbers.	0	-9,223,372,036,854,775,808(-263) to 9,223,372,036,854,775,807(263 -1)

```
public class LongDataType
{
        public static void main(String args[])
        {
                long myLong1, myLong2, result;
                myLong1 = 100000000L;
                myLong2 = 200L;
                result = myLong1 * myLong2;
                System.out.println("Number 1: " +myLong1);
                System.out.println("Number 2: " +myLong2);
                System.out.println("Number 1 * Number 2: " +result);
        }
}
```

OUTPUT

```
Number 1: 100000000
Number 2: 200
Number 1 * Number 2:
20000000000
```

5. Float Data Type

Syntax:
float f = 334.334f;

- The float data type is a single-precision 32-bit IEEE 754 floating point. Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.

Size:	**Values:**	**Default Value:**	**Range:**
4 bytes (32 bits)	Positive or negative decimal numbers.	0.0f	$3.40282347 \times 10^{38}$, $1.40239846 \times 10^{-45}$ Accuracy:- about 7 decimal digit

```
public class floatdemo
{
  public static void main(String[] args)
  {
    float myNum = 5.7335f;
    System.out.println("Float number :"+myNum);
  }
}
```

OUTPUT

```
Float Number : 5.7335
```

6. Double Data Type

Syntax:
double d = 334.3343433;

- The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency.

Size:	Values:	Default Value:	Range:
8 bytes (64 bits)	Positive or negative decimal numbers.	0.0d	1.7E+308 to +1.7E+308 Accuracy:- about 16 decimal digit

```
public class floatdemo
{
  public static void main(String[] args)
  {
    double myNum = 5.73353434;
    System.out.println("Double number :"+myNum);
  }
}
```

OUTPUT

```
Double Number : 5.73353434
```

7. char Data Type

Syntax:
char ch = ‘d’;

- The char data type is a single 16-bit Unicode character. Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive).The char data type is used to store characters.

Size:	**Values:**	**Default Value:**	**Range:**
2 bytes (16 bits)	One characer(or number in char form).	\U0000	From \u0000(0) to \uffff(65535)

```
public class floatdemo
{
  public static void main(String[] args)
  {
    char ch = ‘f’;
    System.out.println(“Value of characer variable
:”+ch);
  }
}
```

OUTPUT

```
Value of character variable : f
```

8. Boolean Data Type

Syntax:
boolean val = true;

- The Boolean data type is used to store only two possible values: true and false. This data type is used for simple flags that track true/false conditions.
- The Boolean data type specifies one bit of information, but its "size" can't be defined precisely

Size:	Values:	Default Value:	Range:
1 bit	True or False	False	True or False

```
public class floatdemo
{
  public static void main(String[] args)
  {
    boolean b = true;
    System.out.println(“Value of boolean variable
:”+b);
  }
}
```

OUTPUT

```
Value of boolean variable :
true
```